

Technical Challenge 2026

概述

任务描述

在本实验中，你需要简单的实现大模型“后训练”的流程。decoder-only架构模型的预训练是让模型在大量文本上学会续写，也就是使用大规模的数据来初始化模型参数，后训练则是设计一系列流程如监督微调SFT、对齐训练Alignment Training。本实验所设计的后训练流水线大致为：

$$SFT \rightarrow RSFT(EI) \rightarrow DPO \rightarrow GRPO(RLV R) \rightarrow \text{Self-play } RL \rightarrow PEFT$$

你需要对Qwen2.5-Math-1.5B进行上述的一系列操作，并分别保存每个阶段完成后的模型

任务要求

本实验考察的是基本的算法实现能力，因此不要使用当前已有的库如Trainer、OptimRL，但是可以使用其他现成的工具如vLLM、Transformers来部署模型和加载模型，以及一些较为基本的深度学习组件如torch.nn.functional等，本实验允许使用AI进行适当的辅助（如解释某个方法的原理），但是**严禁完全借助AI**来完成，也不要照搬网络上已有的实现方案。

由于本实验对超参数非常敏感（即便给定一组超参数也不保证完全复现），本实验不对模型最终的性能做要求，但必须给出一套可运行的代码，并每个阶段完成后模型的性能做对比，如果最终性能较差，给出自己所认为导致性能较差的原因。

最终验收

为方便版本控制以及后续对成果的检查，你需要将实现的代码上传至个人的GitHub仓库，并将仓库设置为公开，每完成一个阶段的子任务或者完成了一整个阶段都需要至少要提交一次代码。

此外，你需要展示你训练过程中的中间信息，比如训练日志、准确率与损失曲线等等，你可以借助wandb或者tensorboard来方便展示。

准备阶段

数据与模型选择

这里给出两个可供选择的数据集，由于gsm8k的难度过低，这里建议使用MATH数据集，（若由于显存问题不得使用更小参数的模型，可以考虑使用gsm8k数据集）

1. gsm8k: <https://huggingface.co/datasets/openai/gsm8k>, 这是openai提供的小学难度的题目, 包含8,500道高质量小学数学应用题 (训练集7,473题, 测试集1,319题), 由标注公司Surge AI制作, 每道题配有详细解法。
2. MATH: https://huggingface.co/datasets/EleutherAI/hendrycks_math, 该数据集包含中小学到高中竞赛级别的数学问题, 难度明显高于GSM8K。

在模型选择方面, 这里指定Qwen2.5-Math-1.5B模型, <https://huggingface.co/Qwen/Qwen2.5-Math-1.5B>, 注意, 不要用Qwen2.5-Math-1.5B-Instruct, 前者是预训练模型, 后者是已经指令微调结束模型。

Baseline

这里将Qwen2.5-Math-1.5B模型的zero-shot性能表现作为baseline, 提示词模板如下

代码块

```
1  # system prompt (please delete this line when copy)
2
3  # Instruction
4  Below is a list of conversations between a human and an AI assistant (you).
5  Users place their queries under "# Query:", and your responses are under "#
   Answer:".
6  You are a helpful, respectful, and honest assistant.
7  You should always answer as helpfully as possible while ensuring safety.
8  Your answers should be well-structured and provide detailed information. They
   should also have an engaging tone.
9  Your responses must not contain any fake, harmful, unethical, racist, sexist,
   toxic, dangerous, or illegal content, even if it may be helpful.
10 Your response must be socially responsible, and thus you can reject to answer
    some controversial topics.
11
12 # Query:
13 ```{instruction}```
14
15 # Answer:
16 ```
```

代码块

```
1  # user prompt (please delete this line when copy)
2
3  A conversation between User and Assistant. The User asks a question, and the
   Assistant solves it. The Assistant first thinks about the reasoning process in
   the mind and then provides the User with the answer. The reasoning process is
   enclosed within <think> </think> and answer is enclosed within <answer>
```

```
</answer> tags, respectively, i.e., <think> reasoning process here </think>
<answer> answer here </answer>.
```

```
4 User: {question}
```

```
5 Assistant: <think>
```

模型的离线推理建议使用vLLM，该推理框架能够极大提升大模型的推理速度，一个vLLM的使用示例如下：

代码块

```
1 from vllm import LLM, SamplingParams
2 from transformers import AutoTokenizer
3
4 tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen3-8B") # 请替换为你实际使用的
   模型路径
5 messages = [
6     {"role": "system", "content": "你是一个由阿里云开发的智能助手，名叫通义千问。"},
7     {"role": "user", "content": "你好，请介绍一下你自己。"}
8 ]
9 # 可以单独打印出prompt，查看运用chat_template之前和之后的区别
10 prompt = tokenizer.apply_chat_template(messages, tokenize=False,
   add_generation_prompt=True)
11
12 llm = LLM(model="Qwen/Qwen3-8B") # 替换为本地路径或模型 ID
13 sampling_params = SamplingParams(temperature=1.0, top_p=1, max_tokens=1024)
14 outputs = llm.generate([prompt], sampling_params)
15
16 for output in outputs:
17     generated_text = output.outputs[0].text
18     print(generated_text)
```

vLLM的推理超参数上述代码已经指定，但是这里需要注意的点是上述设置的prompt要求模型用 `</answer>` 来结束回答，那么这里可以设置vLLM检测到这个 `</answer>` 之后停止生成，防止模型每次都必须回答到长度上限才停止。

代码块

```
1 sampling_params = SamplingParams(
2     temperature=1.0,
3     top_p=1.0,
4     max_tokens=1024,
5     stop=["</answer>"],
6     include_stop_str_in_output=True
7 )
```

到这一步会发现模型的输出并没有如此“听话”，比如问题是“小明有10个苹果，吃了2个，还剩几个”，标准答案是“8”，但大模型可能会回答“8”，也可能回答“小明还剩8个苹果”，甚至可能出现“\boxed{8}个”，那么如何将标准答案与大模型的回答做匹配呢。这里采用deepseek所实现的一个函数来进行多层级的比较，确保尽可能不会“错怪”大模型，你只需要调用 `r1_zero_reward_fn` 即可，其中FAST参数设置为True时，答案匹配速度更快，但错怪大模型的可能性也就越高。

 `drgrpo_grader.py`
29.80KB



SFT

SFT也就是supervised finetuning，本实验所实现的SFT是全量微调，能够对模型的所有参数进行更新。算法伪代码如Algorithm 1所示

Algorithm 1 Supervised Finetuning (SFT)

Require: initial policy model $\pi_{\theta_{init}}$; SFT dataset $\mathcal{D}$

1: policy model $\pi_{\theta} \leftarrow \pi_{\theta_{init}}$

2: **for** step = 1, ..., n_sft_steps **do**

3: Sample a batch of question-response pairs $\mathcal{D}_b$ from $\mathcal{D}$

4: Compute the cross-entropy loss of the responses given the questions using the model π_{θ}

5: Update the model parameters θ by taking a gradient step with respect to the cross-entropy loss

6: **end for**

Ensure: π_{θ}

我们的目标是提升模型的推理能力，而非直接预测正确答案，因此需要微调模型以生成思维链推理轨迹，随后输出答案，也就是最为简单的CoT技术，这种操作能使得模型在不增加模型层数的前提下，显著提升推理性能。

在实际训练推理模型时，通常会先用 SFT 作为第二阶段强化学习（RL）微调的热启动。这主要有两个原因：首先，SFT需要高质量的标注数据（即带有预先存在的推理轨迹），而RL只需正确答案作为反馈；其次，即使在标注数据充足的情况下，RL仍能通过发现优于 SFT 数据的策略来提升性能。

Warm Up

加载HuggingFace模型和tokenizer

示例代码如下所示，不用过多解释

代码块

```
1 from transformers import AutoModelForCausalLM, AutoTokenizer
2
3 model = AutoModelForCausalLM.from_pretrained(
```

```
4     pretrained_model_name_or_path=QWEN_MATH_BASE_PATH,
5     torch_dtype=torch.bfloat16,
6     attn_implementation="flash_attention_2",
7     device_map=device
8 )
9 tokenizer = AutoTokenizer.from_pretrained(QWEN_MATH_BASE_PATH)
```

模型的输入输出

那么问题来了，模型的输入输出是什么，数据集对应的标签在哪？

模型的输入并不是prompt字符串，而是经过tokenizer进行分词后得到的一串“数字列表”，里面的每个数字都表示一个token，将token解码之后得到的概率是“空格+一个英文单词”。然后模型逐个接受这些token，经过前向传播后，输出的也是token，最后输出的token列表经过解码后才得到了我们熟悉的字符串式回答。

聪明的你一定能想到，既然大模型是自回归式的，也就是在做next token prediction，而我们拿到了希望大模型输出的整条数据，是不是只需要将下一个token当成前面的所有token的标签就可以了。没错，全量微调跟大模型的预训练步骤是非常相似的，都是给定 $x_1, x_2, x_3 \dots x_n$ 然后让模型预测 x_{n+1} ，我们拿到的 $\hat{x}_{n+1}$ 就是其标签。

代码块

```
1 data = input_ids[:-1]
2 label = input_ids[1:]
```

梯度累积

由于大显存的显卡非常昂贵，对于NVIDIA GeForce RTX 4090来说，输入的batch_size稍微大一点就会OOM，但是模型输入的batch太小的话，模型的更新次数太多，耗费的时间太长。有没有简单又强势的方法呢，有的，我们接下来要介绍梯度累积技术。

该技术的核心思想是：不要每次处理一个批次就更新模型权重（即 `optimizer.step()` 步骤），而是将梯度在多个批次中累加后再进行梯度更新。直观来说，使用更大容量的GPU一次性处理32个样本的结果，与将样本拆分为16个2样本批次后再取平均值的计算结果是一样的。

那么问题来了，既然梯度累积技术并没有减少梯度所占用的显存空间，那么这个方法是如何降低显存占用的呢，请你进行思考，并且与梯度检查点方法进行比较。

这个在PyTorch中实现起来也很简单，每次执行反向传播后如果不清空梯度，再执行一次反向传播的话，第二次的梯度会累加到原来的梯度上。因此，只需要“攒”够了一整个批次后，再去执行 `optimizer.step()` `optimizer.zero_grad()`。

代码块

```
1 gradient_accumulation_steps = 4
```

```
2  for idx, (inputs, labels) in enumerate(data_loader):
3      logits = model(inputs)
4      loss = loss_fn(logits, labels) / gradient_accumulation_steps
5      loss.backward()
6
7      if (idx + 1) % gradient_accumulation_steps == 0:
8          optimizer.step()
9          optimizer.zero_grad()
```

实现细节注意事项

prompt与response构建

拿到了数据集的question与answer之后，直接把question当作prompt，把answer当作response，这样做虽然可行，但是效果并不会很好。因此，你需要结合之前所给到的prompt模板构建出SFT所用的prompt，并根据answer构建其对应的response，然后将prompt与response编码成token。

此外，还需要构建一个response_mask，其中，response的token对应的位置设为1，prompt的token对应的位置以及padding的位置设为0。这样做是为了确保能够找到response的位置，后续求梯度等步骤只会关注模型的输出（response），因此要消除其余位置的影响。当然，你也可以用更加工程化的方法，构建只包含模型response的句子集合，其余位置使用占位符，从而自动消除了其余位置的影响。

loss计算

另一个角度来讲，大模型预测下一个token的过程其实也是一个多分类的过程，类别的数目等于vocabulary数目，那么这里可以用cross_entropy loss，逐个计算每个token的loss，然后将整个response的loss求和，作为这一条response整体的loss。

当然，你也可以写出其等价形式。首先对模型输出的logits做log_softmax，然后提取出label对应的token的p，即 `p = log_softmax(logits)[:label]`（当然这样写是为了更清楚的展示，实际写的时候注意维度，推荐用 `torch.gather()` 或高级索引方法）。然后进行 `mask_normalize`，也就是只对mask为1的位置进行求平均，最后取反，得到的loss与上面完全等价，但速度要快一点，是工程化的写法。

记录实验数据

目前已经出现了大量的机器学习辅助库，比如wandb、comet等等，可以完全替代tensorboard或者matplotlib，用好这些库能够极大的方便实验数据的管理。

这里推荐几点需要记录的实验数据：

1. 输入prompt与模型的response

2. 真实的answer
3. 准确率、loss的变化
4. response的平均entropy（反应模型置信度或者是否出现过度自信）

测试模型性能

通过huggingface的transformers加载的模型可以通过 `save_pretrained()` 函数来保存模型，在测试阶段可以沿用baseline的代码，修改一下模型路径即可，甚至可以将baseline封装成一个函数，需要测试模型性能的时候调用它，从而做到边进行微调，边查看模型的准确率。

RSFT

前人的研究发现，SFT的数据质量能够显著影响SFT之后模型的性能，因此过滤不良样本能够提升模型性能，在这个前提条件之下，我们引入RSFT。RSFT（Rejection Sampling Fine-Tuning）是一类介于SFT和RL之间的对齐方法，本质上是：用“筛选出来的高质量样本”替代“人工标注数据”，再用标准SFT训练。

算法原理

可以从概率的角度来理解，RSFT实际上在近似优化一个reward-weighted distribution，理想目标是：

$$\pi^*(y|x) \propto \pi_\theta(y|x) \cdot \exp(\beta R(x, y))$$

但RSFT没有显式优化它，而是用“采样 + 截断”来近似：

$$p_{RS}(y|x) \approx \frac{\pi_\theta(y|x) \cdot \mathbf{1}[R(x, y) \geq \tau]}{Z}$$

然后做极大似然估计（MLE）：

$$\max_{\theta} \mathbb{E}_{y \sim p_{RS}} \log \pi_\theta(y|x)$$

算法流程

算法流程是很简单的，只需要先进行采样，得到G个输入-输出对，然后用奖励函数R来计算每对 `(q_i, o_i)` 的奖励，筛选出奖励超过一定阈值的样本，然后用这些样本来进一步SFT（当然，你也可以使用Top-K或者Top-P来筛选这G个样本）算法伪代码在下面已经给出

Algorithm 2 Rejection Sampling Fine-Tuning (RSFT)

Require: initial policy model $\pi_{\theta_{\text{init}}}$; reward function R ; task questions $\mathcal{D}$

- 1: policy model $\pi_{\theta} \leftarrow \pi_{\theta_{\text{init}}}$
- 2: **for** step = 1, ..., n_rsft_steps **do**
- 3: Sample a batch of questions $\mathcal{D}_b$ from $\mathcal{D}$
- 4: Set the old policy model $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$
- 5: Sample G outputs $\{o^{(i)}\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q)$ for each question $q \in \mathcal{D}_b$
- 6: Compute rewards $\{r^{(i)}\}_{i=1}^G$ for each sampled output $o^{(i)}$ by running reward function $R(q, o^{(i)})$
- 7: Filter correct outputs $\mathcal{D}_{\text{sft}} = \{(q_i, o_i) \mid r_i \geq \tau\}$ to obtain a dataset $\mathcal{D}_{\text{sft}}$ of better-quality question-response pairs
- 8: $\pi_{\theta} \leftarrow \text{SFT}(\pi_{\theta}, \mathcal{D}_{\text{sft}})$
- 9: **end for**

Ensure: π_{θ}

在构建正确样本的时候，可以对vLLM的SamplingParams进行进一步的修改，添加参数 `n=G` 使得大模型对于一个prompt来生成G个response，添加 `min_tokens` 参数确保大模型不会生成空字符串。相应的修改如下：

代码块

```
1  sampling_params = SamplingParams(  
2      temperature=1.0,  
3      top_p=1.0,  
4      max_tokens=1024,  
5      min_tokens=min_tokens,  
6      n=G,  
7      stop=["</answer>"],  
8      include_stop_str_in_output=True,  
9      seed=seed # optional  
10 )
```

DPO

现在已经进入了强化学习的阶段，DPO属于比较特殊的强化学习方法，该算法在PPO的基础之上进行了大幅度简化，不需要额外的奖励模型，也属于是实现非常简单的一种方式。

理论推导

RLHF的优化目标

我们从标准的 RLHF 目标出发：

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi(\cdot|x)} [r(x, y)] - \beta \cdot \text{KL}(\pi(\cdot|x) | \pi_{\text{ref}}(\cdot|x))$$

展开 KL散度：

$$\text{KL}(\pi|\pi_{\text{ref}}) = \mathbb{E}_{y \sim \pi} \left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \right]$$

代入目标：

$$\max_{\pi} \mathbb{E}_{y \sim \pi} \left[r(x, y) - \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \right]$$

DPO最优策略

因为不同 prompt 独立，可以对固定 x 求最优策略：

$$\max_{\pi(\cdot|x)} \sum_y \pi(y|x) \left[r(x, y) - \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \right] \quad s.t. \quad \sum_y \pi(y|x) = 1$$

接下来用拉格朗日法求最优策略，构造拉格朗日函数（也就是将一个约束优化问题变成无约束优化）

$$\mathcal{L} = \sum_y \pi(y|x) \left[r(x, y) - \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \right] + \lambda \left(\sum_y \pi(y|x) - 1 \right)$$

对 $\pi(y|x)$ 进行求导，注意到 $\frac{d}{d\pi} (\pi \log \pi) = \log \pi + 1$ ，所以

$$\frac{\partial \mathcal{L}}{\partial \pi(y|x)} = r(x, y) - \beta \left(\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} + 1 \right) + \lambda$$

令导数为 0：

$$r(x, y) - \beta \left(\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} + 1 \right) + \lambda = 0$$

整理：

$$\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} = \frac{1}{\beta} (r(x, y) + \lambda') \quad (\text{常数吸收: } \lambda' = \lambda - \beta)$$

因此：

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp \left(\frac{1}{\beta} r(x, y) \right)$$

也就是说，DPO最优策略的形式为以下公式

$$\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \cdot \exp \left(\frac{1}{\beta} r(x, y) \right)$$

推导DPO损失函数

对上式取 log

$$\log \pi^*(y|x) = \log \pi_{\text{ref}}(y|x) + \frac{1}{\beta} r(x, y) - \log Z(x)$$

整理：

$$r(x, y) = \beta [\log \pi^*(y|x) - \log \pi_{\text{ref}}(y|x)] + \text{const}$$

下面需要引入偏好数据，我们没有 $r(x, y)$ ，但有偏好 $y_w \succ y_l$ （记住DPO是Direct Preferences Optimization）

引入Bradley-Terry 模型

$$P(y_w \succ y_l) = \sigma(r(x, y_w) - r(x, y_l))$$

代入 reward 表达式，得到likelihood

$$P(y_w \succ y_l) = \sigma \left(\beta \left[\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right] \right)$$

接下来用最大似然来推导出DPO Loss，最大化 $\log P(y_w \succ y_l)$ 得到：

$$\mathcal{L}_{DPO} = -\log \sigma \left(\beta \left[\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right] \right)$$

这也就是DPO的损失函数

实现细节注意事项

reference模型选取

现在问题来了，reference模型如何选取，直接观察“奖励模型”可以发现

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \text{const}$$

r 最终的效果其实是 $\pi^*(y|x)$ 相对于 $\pi_{\text{ref}}(y|x)$ 提升的程度。如果选用更强的模型如DeepSeek-V3作为reference模型的话，会发现模型输出分布不一致，没有任何的可比性，因为更强的模型在SFT以及后续阶段所使用的数据集、训练时间、优化目标与我们前面所训练的模型不一致。

那么既然没办法跟其他模型“内卷”，该怎么判断是否“进步”呢。答案很显然，只需要比“以前的自己”强就好了，那么这里所选取的reference模型就是冻结参数的完成RSFT之后的模型。

log_prob计算

你可能会想，既然模型是自回归式的生成token，后面生成的token需要依赖前面的，那直接取 `logits[:-1:label]` 做 `log_softmax` 当作是最终的 $\pi(y|x)$ 不就好了，当然，这是不对的。假如模型的response长度为t，那么可以得到

$$\pi(y|x) = p(y_1|x) \cdot p(y_2|x, y_1) \dots p(y_t|x, y_{<t}) = \prod_t p(y_t|x, y_{<t})$$

注意这里的p是logits进行softmax之后得到的概率，上式取对数可得

$$\log \pi(y|x) = \log p(y_1|x) + \log p(y_2|x, y_1) \dots + \log p(y_t|x, y_{<t}) = \sum_t \log p(y_t|x, y_{<t})$$

这下非常的清晰明了，原来只需要将每个位置的log_softmax相加就可以得到 $\pi(y|x)$ ，值得注意的是，这里相加的log_softmax对应位置的也是response，需要写一个掩码或者记录一下response的起

止下标。

DPO loss的伪代码

代码块

```
1  logp_w = log_prob(model, x, y_w)
2  logp_l = log_prob(model, x, y_l)
3
4  logp_w_ref = log_prob(ref_model, x, y_w)
5  logp_l_ref = log_prob(ref_model, x, y_l)
6
7  loss = -logsigmoid(
8      beta * ((logp_w - logp_w_ref) - (logp_l - logp_l_ref))
9  )
```

β 的选取

这里引用某篇知乎上的结论：

- 较大 β （如 1.0）：放大 reward 或 logp 的差异，使模型更“自信”地倾向于较优样本，但容易过拟合或 reward 震荡。
- 较小 β （如 0.1）：差异被压缩，模型训练更稳定，但收敛较慢、辨别力较弱。
- 极小 β （趋近于 0）：差异几乎无效，模型无法区分好坏样本，退化为随机训练

较小的 β 会使得训练过程更加接近SFT，较大的 β 会让模型过于“有野心”，因此，建议这个值设置在0.1~0.5之间。

GRPO

刚刚的DPO只是热身，现在才真正进入了RL的领域。以下的推导涉及到了部分RL专有名词，比如 trajectory、policy(agent)、advantage、state(environment)、action、critic(value)，建议先整体了解一下RL的基本原理，这里推荐B站李宏毅老师的机器学习课程，以及这节课的笔记

【(强推)李宏毅2021/2022春机器学习课程】 https://www.bilibili.com/video/BV1Wv411h7kN/?p=114&share_source=copy_web&vd_source=dac4be499f7f39cb7e2f15971fdef951
<https://diamond-mule-bee.notion.site/12-Reinforcement-Learning-RL-00c5c8c0e77749e3add1726525467ff5>

如果你了解过DeepSeek R1的原理，你应该对GRPO并不陌生，本章节所涉及到的GRPO也被归类于RLVR（Reinforcement Learning with Verified Rewards），这个方法能够极大提升模型推理能力。类似于DPO，deepseek的方法也将reward model也给“优化”掉，不用单独去训练一个额外的模型，并且还取得了更优的性能。

理论推导

大模型场景下的RL组件定义

给定输入前缀 s_t ，大模型会得到下一个输出token $a_t \in \mathcal{V}$ 的概率分布，在RL的场景之下，可以将选择的下一个token a_t 当作是action，把输入前缀 s_t 当作state，那么可以把大模型当成一个随机性分类策略(categorical stochastic policy)。

$$a_t \sim \pi_{\theta}(\cdot | s_t), \pi_{\theta}(\cdot | s_t) = [\text{softmax}(f_{\theta}(s_t))]$$

现在定义了action，接下来定义state，初始的state s_0 是从一个初始的分布中采样得到的 $s_0 \sim \rho_0(s_0)$ ，其中 $\rho_0(s_0)$ 为模型prompt的一个概率分布，那么很显然，后续的state其实就是prompt拼接后面所生成的token，也就是说

$$s_{t+1} = s_t || a_t$$

这跟一般情况下的state定义 $s_{t+1} \sim P(\cdot | s_t, a_t)$ 的确有所不同，但是这么搞似乎更简单了。那么定义的trajectory也就显而易见，假设T是模型response的长度，那么trajectory τ 为

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T, a_T)$$

定义RL的目标

对于一般场景， $R(\tau)$ 的定义可以为前面所有步骤的累积奖励

$$R(\tau) := \sum_{t=0}^T r_t \text{ 或者 } R(\tau) := \sum_{t=0}^T \gamma^t r_t$$

RL的目标是最大化这个 $R(\tau)$ ，但是在有可验证奖励的RL场景（RLVR），如果没走到最后一步，是不会产生任何奖励的（完成一个正确步骤给一个步骤奖励的叫做PRM RL——Process Reward Model，与本实验的思想不一致），也就是说 $R(\tau)$ 可以有以下定义

$$R(\tau) = \sum_{t=0}^T r_t = r_T := \begin{cases} 1 & \text{如果奖励模型验证trajectory为真} \\ 0 & \text{其他情况} \end{cases}$$

然后agent的目标就是最大化 $R(\tau)$ 的期望

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$$

然后剩下就是一个优化（optimization）问题了

$$\theta^* = \arg \max_{\theta} J(\theta)$$

Policy Gradient

上述的优化问题可以用梯度上升来解决（你可能会听过梯度下降，这个是最小化Loss时用的，现在我们的目标是最大化，当然加个负号就又变成梯度下降了）

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} J(\pi_{\theta_k})$$

然后我们接下来尝试求解梯度 $\nabla_{\theta} J(\pi_{\theta})$

首先，如果给定了参数 θ ，那么得到trajectory τ 的概率为

$$P(\tau | \theta) = \rho_0(s_0) \prod_{t=0}^T P(s_{t+1} | s_t, a_t) \pi_\theta(a_t | s_t)$$

然后对上述的概率分布求对数

$$\log P(\tau | \theta) = \log \rho_0(s_0) + \sum_{t=0}^T [\log P(s_{t+1} | s_t, a_t) + \log \pi_\theta(a_t | s_t)]$$

根据log函数的微分公式可以得到

$$\nabla_\theta \log P = \frac{1}{P} \nabla_\theta P \quad \text{i.e.} \quad \nabla_\theta P = P \nabla_\theta \log P$$

很多预先定义好的组件不会随着模型参数 θ 的变化而变化，比如 $\rho_0(s_0)$, $P(\cdot|\cdot)$, $R(\tau)$

$$\nabla_\theta \rho_0 = \nabla_\theta P = \nabla_\theta R(\tau) = 0$$

然后把这些规律统统带入下面的公式

$$\begin{aligned} \nabla_\theta J(\theta) &= \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] \\ &= \nabla_\theta \sum_{\tau} P(\tau|\theta) R(\tau) \\ &= \sum_{\tau} \nabla_\theta P(\tau|\theta) R(\tau) \\ &= \sum_{\tau} P(\tau|\theta) \nabla_\theta \log P(\tau|\theta) R(\tau) \quad (\text{log微分公式}) \\ &= \mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \log P(\tau|\theta) R(\tau)] \quad (\text{期望的数学定义}) \\ &= \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) R(\tau) \right] \quad (\text{消掉微分为0的项}) \end{aligned}$$

也就是说

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) R(\tau) \right]$$

假如有N个rollouts $\mathcal{D} = \{\tau^{(i)}\}_{i=1}^N$ ，每个rollouts的初始state都是从prompt构造的分布采样得到的 $s_0^{(i)} \sim \rho_0(s_0)$ ，然后跑了policy N次。这样可以构建一个梯度的无偏估计

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)}) R(\tau^{(i)})$$

Policy Gradient Baseline

现在是得到了梯度，虽然已经能直接使用，但是梯度的方差很高，训练不稳定，一个很常规的想法是进行归一化。所以要在 $R(\tau)$ 的基础上减去一个baseline，那么问题来了，减去一个baseline之后，原先的梯度还能不能用。用公式来说明的话，减去baseline的policy gradient可以写成下面的形式

$$B = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (R(\tau) - b(s_t)) \right]$$

如果把advantage $A(\tau)$ 定义为 $A(\tau) = R(\tau) - b(s_t)$ ，那么上式也可以写成

$$B = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A(\tau) \right]$$

一个合理的b是value函数，也就是 $R(\tau)$ 的期望值 $b(s) = V^{\pi}(s) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau) | s_t = s]$ ，从而advantage就是表示这个动作比平均水平高了多少。

将 B 展开，可以写出下面的公式

$$B = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right] - \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) b(s_t) \right]$$

将第二项进行变换，得到

$$\mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) b(s_t) \right] = \sum_{t=0}^T \mathbb{E}_{s_t} [b(s_t) \mathbb{E}_{a_t \sim \pi_{\theta}(\cdot | s_t)} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]]$$

观察拆分的最后一项，根据 $\mathbb{E}_{x \sim P_{\theta}} [\nabla_{\theta} \log P_{\theta}(x)] = 0$ （可以自己证明，把期望写成积分或求和形式），可以发现，上面的公式的值为0，那么

$$B = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right] - 0 = \nabla_{\theta} J(\pi_{\theta})$$

因此，即使引入了baseline，梯度的期望还是不变的，没有违反RL的优化目标。那么，引入baseline之后，梯度的一个无偏估计可以写成以下形式

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) (R(\tau^{(i)}) - b(s_t^{(i)}))$$

离线policy gradient

上述所提到的强化学习是一个“在线”的算法，训练资料是从正在被优化的模型中得到的，也就是说，收集一次资料需要非常多的时间却只能跑一轮梯度更新，于是需要用离线的算法。

对于离线的RL算法，没必要非要从正在被优化的模型中收集资料，而是说收集的一次资料可以跑多轮梯度更新，但是需要根据模型输出进行调整，也就是进行了“重要性采样”。假设当前的模型为 π_{θ} ，收集资料时的模型为 π_{old} ，那么离线的一个梯度估计为

$$\hat{g}_{\text{off-policy}} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \frac{\pi_{\theta}(a_t^{(i)} | s_t^{(i)})}{\pi_{\theta_{\text{old}}}(a_t^{(i)} | s_t^{(i)})} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) (R(\tau^{(i)}) - b(s_t^{(i)}))$$

详细的推导过程可以参考PPO，这里不再赘述。

loss计算

上面说了半天，都是在说梯度的估计，具体实施的时候难道要手动传进去梯度来实现梯度下降吗。当然没那个必要，我们只需要主动定义一个loss，然后调用 `loss.backward()` 让pytorch自动反向传播就行了。当然，这里所定义的loss没有任何实际的含义，只是方便我们进行反向传播的一个“跳板”，loss的定义也比较简单，把上式进行“积分”便可以得到（注意log怎么消失了，想一下变量 θ ，以及log微分公式 $\nabla_{\theta} P = P \nabla_{\theta} \log P$ ），原来的目标是梯度上升，现在搞成了loss变成了梯度下降，所以要在前面加个负号

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \frac{\pi_{\theta}(a_t^{(i)} | s_t^{(i)})}{\pi_{\theta_{\text{old}}}(a_t^{(i)} | s_t^{(i)})} (R(\tau^{(i)}) - b(s_t^{(i)}))$$

当然，如果定义了advantage $A(\tau) = R(\tau) - b(s_t)$ ，概率比 $r_t(\theta) = \frac{\pi_{\theta}(a_t^{(i)} | s_t^{(i)})}{\pi_{\theta_{\text{old}}}(a_t^{(i)} | s_t^{(i)})}$ 也可以进一步简化

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T r_t(\theta) A(\tau^{(i)})$$

到了现在你可能会非常懵了，我们所得到的这个“假的损失函数”到底有什么用，回顾上面的过程，你会发现，我们的目的是最大化奖励的期望 $J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$ ，但是这个东西涉及采样，所以我们才对它进行求导 $\nabla_{\theta} J(\theta)$ ，然后中途引入了baseline，但是我们证明了即使引入了baseline也不会改变 $\nabla_{\theta} J(\theta)$ ，然后为了方便书写，我们用 $\hat{g}$ 来代表 $\nabla_{\theta} J(\theta)$ ，之后发现得到 $\hat{g}$ 的代价太高了，于是引入离线机制令 $\hat{g}_{\text{off-policy}} \approx \hat{g}$ ，最后重新对 $\hat{g}_{\text{off-policy}}$ 进行积分得到了 $J(\theta)$ 的一个近似的负值 $\mathcal{L}(\theta)$ ，因此，我们只需要最小化 $\mathcal{L}(\theta)$ 就能够最大化 $J(\theta)$ 。

clip机制

当概率比 $r_t(\theta)$ 的波动非常大的时候，策略的更新就会非常大（好牌，全部拿走，坏牌，全部丢掉），训练也会十分不稳定，为了解决这个问题，需要用clip机制来限制更新幅度。那么，如果只是重新把损失写成

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A(\tau^{(i)})$$

观察这个公式，当模型认为动作优势（ $A > 0$ ）非常大的时候（ r 很大），赶紧给他“拽回”别太激动，这没问题，但是，当模型认为优势较小的时候（ r 很小），有必要帮模型假装他想多做这件事吗，同样的，当模型认为劣势（ $A < 0$ ）很大，于是想要完全不去做这件事（ r 很小），但是我们必须让他循序渐进地来，不要完全不去做，这没问题，但是当模型想继续错下去（ r 很大），我们有必要帮模型假装他想少做吗。上面的话有点绕，可以多思考一下，当你思考明白了会发现直接clip会有问题，虽然消除了一半不良影响，却增加了一半本来不存在的不良影响。

那么，为了解决这些问题，需要重新对公式进行修正，修正后的公式如下

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \min \left(r_t(\theta) A(\tau^{(i)}), \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A(\tau^{(i)}) \right)$$

这样再进行分类讨论的话，就会发现clip导致的bug全修了，这正是PPO的厉害之处。

GRPO

所以GRPO干了什么，实际上GRPO只是把PPO的advantage A改了改，原来的advantage是 $A(\tau) = R(\tau) - b(s_t)$ ，其中b可能是value函数或者其他，现在是

$$A(\tau^{(i)}) = \frac{r^{(i)} - \text{mean}(r^{(1)}, r^{(2)}, \dots, r^{(G)})}{\text{std}(r^{(1)}, r^{(2)}, \dots, r^{(G)}) + \text{advantage_eps}}$$

甚至后面有研究发现，没必要写成这种“归一化”的形式，直接写成下面这种就行

$$A(\tau^{(i)}) = r^{(i)} - \text{mean}(r^{(1)}, r^{(2)}, \dots, r^{(G)})$$

解释一下r与G是什么意思，其实就是对于同一个问题q，生成了一组G个输出，然后对于第i个输出的奖励 $r^{(i)} = R(q, o^{(i)})$ ，后面的平均值充当了一个baseline的作用，而且还是朴实无华的求平均。但就是如此简单的改动，却起到了显著的效果。

实现细节注意事项

advantage计算

这里重新声明一下advantage的求解公式

$$A(\tau^{(i)}) = r^{(i)} - \text{mean}(r^{(1)}, r^{(2)}, \dots, r^{(G)})$$

其中G是对于同一个问题所生成的答案数量，i是一组答案中答案的序号，对于同一个问题q生成G个回答的方法在上文也已经提到，就是把SamplingParams的参数n设置为组数G即可。

GRPO loss

损失函数的计算公式为

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \min \left(r_t(\theta) A(\tau^{(i)}), \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A(\tau^{(i)}) \right)$$

其中， $r_t(\theta) = \frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{\text{old}}}(o_t|q, o_{<t})}$ ，也就是输出的logits进行softmax之后得到 $\pi_{\theta}(\cdot)$ ，再把N替换成G，然后注意T是输出o的长度，即 $T = |o^i|$ 。你可能又会想，新旧模型参数不一致，那同样的prompt输出也不一样啊，直接比较概率有意义吗，再仔细观察上面公式，你会发现就是要让两个模型有同样的state，同样的输出token，然后再比较概率，这里跟计算advantage的时候作出区分。注意一下，如果你是按照逐个token相加的方式来计算损失，你会发现A似乎少了t的下标，没关系，每个token位置的A的值可以用整个trajectory的A的值来替代，不影响结果。整个算法的伪代码为

Algorithm 3 Group Relative Policy Optimization (GRPO)

Require: initial policy model $\pi_{\theta_{\text{init}}}$; reward function R ; task questions $\mathcal{D}$

```
1: policy model  $\pi_{\theta} \leftarrow \pi_{\theta_{\text{init}}}$ 
2: for step = 1, ..., n_grpo_steps do
3:   Sample a batch of questions  $\mathcal{D}_b$  from  $\mathcal{D}$ 
4:   Set the old policy model  $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ 
5:   Sample  $G$  outputs  $\{o^{(i)}\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q)$  for each question  $q \in \mathcal{D}_b$ 
6:   Compute rewards  $\{r^{(i)}\}_{i=1}^G$  for each sampled output  $o^{(i)}$  by running re-
     ward function  $R(q, o^{(i)})$ 
7:   Compute  $A^{(i)}$  with group normalization
8:   for train step = 1, ..., n_train_steps_per_rollout_batch do
9:     Update the policy model  $\pi_{\theta}$  by maximizing the GRPO-Clip objective
10:  end for
11: end for
```

Ensure: π_{θ}

超参数推荐

代码块

```
1  n_grpo_steps: int = 200
2  learning_rate: float = 1e-5
3
4  advantage_eps: float = 1e-6
5  rollout_batch_size: int = 32
6  group_size: int = 8
7  sampling_temperature: float = 1.0
8  sampling_min_tokens: int = 4
9  sampling_max_tokens: int = 1024
10 epochs_per_rollout_batch: int = 2
11 train_batch_size: int = 8
12 gradient_accumulation_steps: int = 4
13 gpu_memory_utilization: float = 0.85
14 optimizer = torch.optim.AdamW(
15     policy.parameters(),
16     lr=learning_rate,
17     weight_decay=0.0,
18     betas=(0.9, 0.95),
19 )
```

Self-Play RL

概念与动机

自对弈。 在博弈论与强化学习中，**自对弈 (self-play)** 指智能体与自身或自身的历史版本对弈，从而在没有外部对手的情况下自我提升。AlphaGo、AlphaZero 等通过自对弈在围棋等游戏中达到超人类

水平。

自对弈与语言模型。 近期工作将自对弈引入语言模型：有在形式化数学证明中采用 AlphaZero 风格训练的；有让模型在零和博弈中与不断改进的自身对弈的；也有采用非对称自对弈，由 proposer 生成问题、solver 解答，两者通过 RL 共同提升的。这些工作显示，**模型自己出题、自己解题** 能形成可迁移的推理模式，并在无外部标注数据的情况下自我提升。

问题生成 + 自解

本任务采用 **问题生成 + 自解** 的 self-play 方案：模型同时扮演「出题者」与「解题者」，先生成数学问题及标准答案，再尝试解答自己出的题，用验证器判定正确性，并将正确样本用于训练。该方案真正体现自对弈——模型与自身（出题角色 vs 解题角色）交互。

流程概览：

- 出题阶段：**用 prompt 引导模型生成「问题 + 标准答案」对，例如要求输出 `Problem: ...`
`Answer: ...` 格式。
- 解题阶段：**对生成的问题，用 `r1_zero` prompt 让模型生成带思维链的解答。
- 验证阶段：**用 `r1_zero_reward_fn` 或 `drgrpo_grader.grade` 比较解题输出与出题时附带的标准答案，判定正确与否。
- 训练阶段：**正确样本可加入 SFT 数据做监督微调，或作为 GRPO 的 reward 信号（正确=1，错误=0）进行策略梯度更新。

实现要点与代码提示

问题生成 Prompt 设计

模型需同时生成问题和答案，以便后续验证。可设计如下 prompt 模板：

代码块

```
1  PROBLEM_GEN_PROMPT = """You are a math problem creator. Generate a math word
   problem that can be solved with arithmetic or algebra, and provide the final
   numerical or symbolic answer.
2
3  Output in the following format exactly:
4  Problem: [Your math problem here, in natural language]
5  Answer: [The final answer, e.g. 42 or \frac{1}{2} or x^2 + 1]
6
7  Generate one problem now: """
8
9  # 或基于 MATH 种子问题做变体生成
10 SEED_PROMPT_TEMPLATE = """Here is a math problem: {seed_problem}
11
12 Create a new, different math problem that is similar in difficulty and topic.
   Output:
```

```
13 Problem: [new problem]
14 Answer: [answer]"""
```

生成时使用较高 temperature（如 0.8–1.0）以增加多样性，`max_tokens` 建议 256–512。

解析问题与答案

从模型输出中解析出 `problem` 和 `answer`，用于后续解题与验证：

代码块

```
1 import re
2
3 def parse_problem_and_answer(gen_output: str) -> tuple[str | None, str | None]:
4     """从问题生成输出中解析 Problem 和 Answer。"""
5     problem_match = re.search(r"Problem:\s*(.+?)(?=Answer:|$)", gen_output,
6                               re.DOTALL)
7     answer_match = re.search(r"Answer:\s*(.+?)$", gen_output, re.DOTALL)
8     problem = problem_match.group(1).strip() if problem_match else None
9     answer = answer_match.group(1).strip() if answer_match else None
10    return problem, answer
```

需处理解析失败的情况（返回 None 时跳过该样本）。

解题与验证

对解析出的问题，用 `r1_zero` prompt 格式化后让模型解题，再用 `r1_zero_reward_fn` 验证（需传入解析出的 `answer` 作为 `ground_truth`）：

代码块

```
1 from drgrpo_grader import r1_zero_reward_fn
2
3 # 格式化解题 prompt
4 def format_solve_prompt(problem: str) -> str:
5     with open("cs336_alignment/prompts/r1_zero.prompt") as f:
6         template = f.read()
7         return template.replace("{question}", problem)
8
9 # 解题
10 solve_prompts = [format_solve_prompt(p) for p, _ in problems_with_answers]
11 solve_outputs = llm.generate(solve_prompts, sampling_params)
12
13 # 验证: ground_truth 来自出题阶段解析的 answer
14 for i, (problem, gt_answer) in enumerate(problems_with_answers):
15     response = solve_outputs[i].outputs[0].text
```

```
16 reward_info = r1_zero_reward_fn(response, gt_answer)
17 is_correct = reward_info["answer_reward"] == 1.0
```

注意：`r1_zero_reward_fn` 要求 `response` 含 `</think>`、`<answer>` 等标签；若问题生成格式不同，可改用 `drgrpo_grader.grade` 直接比较解析出的答案与 `ground_truth`。

Self-Play 主循环

将出题、解题、验证、训练串联成循环：

代码块

```
1 def self_play_step(policy, llm, problem_gen_prompt, n_problems_per_step=64):
2     # 1. 出题: 生成 n_problems_per_step 个 (problem, answer) 对
3     gen_prompts = [problem_gen_prompt] * n_problems_per_step
4     gen_outputs = llm.generate(gen_prompts, problem_gen_sampling_params)
5     problems_with_answers = []
6     for out in gen_outputs:
7         problem, answer = parse_problem_and_answer(out.outputs[0].text)
8         if problem and answer:
9             problems_with_answers.append((problem, answer))
10
11     if not problems_with_answers:
12         return # 本步无有效问题, 跳过
13
14     # 2. 解题: 用 r1_zero prompt 让 policy 解答
15     load_policy_into_vllm_instance(policy, llm)
16     solve_prompts = [format_solve_prompt(p) for p, _ in problems_with_answers]
17     solve_outputs = llm.generate(solve_prompts, solve_sampling_params)
18
19     # 3. 验证并收集正确样本
20     correct_pairs = []
21     for i, (_, gt) in enumerate(problems_with_answers):
22         resp = solve_outputs[i].outputs[0].text
23         if r1_zero_reward_fn(resp, gt)["answer_reward"] == 1.0:
24             correct_pairs.append((solve_prompts[i], resp))
25
26     # 4. 训练: 可用 SFT 在 correct_pairs 上微调, 或构造 GRPO rollout 用 reward 更新
27     if correct_pairs:
28         # 选项 A: SFT
29         train_sft_on_pairs(policy, correct_pairs, ...)
30         # 选项 B: GRPO (将 correct_pairs 视为 reward=1 的 rollout, 错误样本为
reward=0)
31         # ...
```

与 GRPO 结合

若采用 GRPO 而非 SFT 更新：对每个生成的问题，采样多个解答（ $n=G$ ），用解析出的 answer 作为 ground_truth 计算 reward，再按 GRPO 流程计算 advantage 并更新 policy。此时 self-play 提供「自产」的 (prompt, response, reward) 三元组，无需外部 MATH 数据。

注意事项

问题质量：生成的问题可能无解、答案错误或难以解析，需过滤无效样本。

课程学习：可先用 MATH 中的简单题作为种子，让模型生成变体，再逐步放开。

评估：仍在 MATH 验证集上评估，self-play 数据仅用于训练，不参与验证指标计算。

交付物：Self-Play 脚本（含问题生成、解析、解题、验证、训练流程）、验证准确率曲线、问题生成格式与过滤策略的简要设计说明。

PEFT

在Self-Play之后，你已经获得了一个指令微调好的大模型。但是，这个大模型是基于Math数据集进行训练的，对其他领域的问题，比如做物理题或者写作文等问题可能并不擅长。为此，我们要对大模型进行跨领域微调，使其能够执行不同的任务。

然而，在模型训练过程中，每次更新权重都要对整个权重矩阵进行更新。形式化地说，对权重矩阵 $W \in \mathbb{R}^{d \times k}$ 在每次权重更新时，执行 $W \leftarrow W + \Delta W$ 。这样，每次更新时都要对 $d \times k$ 个参数进行更新，在现代十亿级别参数的大模型训练中对显存和算力提出了极高的要求。

为了解决这一问题，LoRA应运而生。LoRA通过对权重矩阵的低秩分解，将需要更新的 $d \times k$ 个参数降维到 $d \times r$ 和 $r \times k$ 两个较小的矩阵，实现了高效的微调过程。

LoRA

可以阅读原论文：<https://arxiv.org/abs/2106.09685>

低秩分解

如果你对线性代数的内容还有印象，那么对于一个矩阵 M ，可以将其分解为两个矩阵 A 和 B ，使得 $M = A \times B$ 。LoRA正是利用了这一性质，将权重更新中的 ΔW 矩阵替换成了 A 和 B 两个矩阵，并令 $\Delta W = A \times B$ 。这样，只需要训练 A 和 B 两个矩阵，就能获得 ΔW 矩阵并更新权重。

为什么这样可行？

Aghajanyan 等人的研究表明，预训练模型拥有极小的**内在维度**(intrinsic dimension)，即存在一个极低维度的参数，微调它和在全参数空间中微调能起到相同的效果，且模型参数越多，内在参数的维度就越低。LoRA认为，参数更新过程中也存在一个‘内在秩’。因此，对权重矩阵的低秩分解是有效的。

LoRA的执行过程

在LoRA中，我们用矩阵A和B来近似表达 ΔW 。设 $W \in \mathbb{R}^{d \times k}$ ，那么有 $A \in \mathbb{R}^{d \times r}$ ， $B \in \mathbb{R}^{r \times k}$ ，其中 r 被称为秩。根据原论文，矩阵A初始化为0，B随机初始化或高斯初始化。同时引入一个超参数 α ，这样，每次更新时可按照如下公式：

$$W' = W + \frac{\alpha}{r} AB$$

例如在LoRA源码对GPT2微调，做NLG任务时，就取 α 为32， r 为4。

对LoRA更详细的解释

任务要求

实现一个LoRA模块，不要调用现有的、已经实现好的LoRA有关库，对Qwen2.5-Math-1.5B进行微调，使其能够解决物理问题。

数据集选择

你可以使用这个数据集：<https://huggingface.co/datasets/Cloudriver/PhyX>

也可以使用这个数据集：<https://huggingface.co/datasets/nyu-ml/glu>

还可以使用其他你觉得合理的数据集，并不对数据集进行特定要求，当然，不能使用数学领域的数据集。

低秩矩阵位置

提示：Microsoft的人尝试了一些值，发现偶数和 $r = 1$ 的矩阵表现出奇地好。当数据与预训练中使用的数据相似时，低 r 值可能就足够了。

此外，根据原论文的实验效果来看，同时对transformer的q、k、v部分采用LoRA的效果更好。你可以尝试print这个模型，即`model.named_children()`或者`model.named_module()`，然后你会看到模型attention的q、k、v、o参数以及FFN的参数名称，然后想办法冻结原始模型参数只训练低秩矩阵。

出于代码复杂程度的考虑，你只需要将LoRA运用到FFN层即可（`down_proj`、`up_proj`、`gate_proj`）。